



Guestbook App

Sign In, Stay Connected – Your Thoughts, Live and Interactive with Our Guestbook App

Goal

The goal of this project was to develop a full-stack Guestbook application using Node.js and Express, allowing users to add and view messages on a public guestbook page. The objective was to create a seamless and dynamic user experience by implementing AJAX for real-time message updates, while ensuring the front-end design was clean and responsive with Bootstrap. Additionally, the project aimed to demonstrate proficiency in handling asynchronous operations, data storage using JSON, and deploying the application on a cloud platform.

Key Features & Functionality:

Web Server Setup with Node.js and Express:

A Node.js server was set up using Express to efficiently handle routing and HTTP requests.

Four essential routes were implemented to manage the flow of data within the application:

- **/ (Home Route):** The site's main webpage, customizable for various purposes (e.g., company introduction or greeting page). It includes navigation links to other sections of the app, providing easy access to all features.
- **/guestbook:** This route retrieves the guestbook entries stored in a JSON file and displays them in a well-organized table, styled using Bootstrap (with an optional Pure.css alternative for styling).
- **/newmessage:** This page features a user-friendly input form, where users can submit their name, country, and message. A validation process ensures that all fields are completed before submission.
- **/ajaxmessage:** This route allows users to submit a message asynchronously using JavaScript, eliminating the need for a traditional form submission. The data is sent to the backend via AJAX, and the page is dynamically updated with the latest guestbook entries without needing a page refresh.

Message Handling and Storage:

When a user submits a message, the data—including a timestamp generated with JavaScript's `Date()` function—is saved to a JSON file. While the ID is optional, it can be generated for each entry if required, ensuring that each message is uniquely identifiable.

AJAX Implementation:

The AJAX route (`/ajaxmessage`) enables a smooth, interactive experience by sending and receiving data asynchronously. Upon submission, JavaScript captures the user's input and sends it to the backend. The server responds with the updated guestbook data, and the page refreshes dynamically to display all messages in real time.

Deployment:

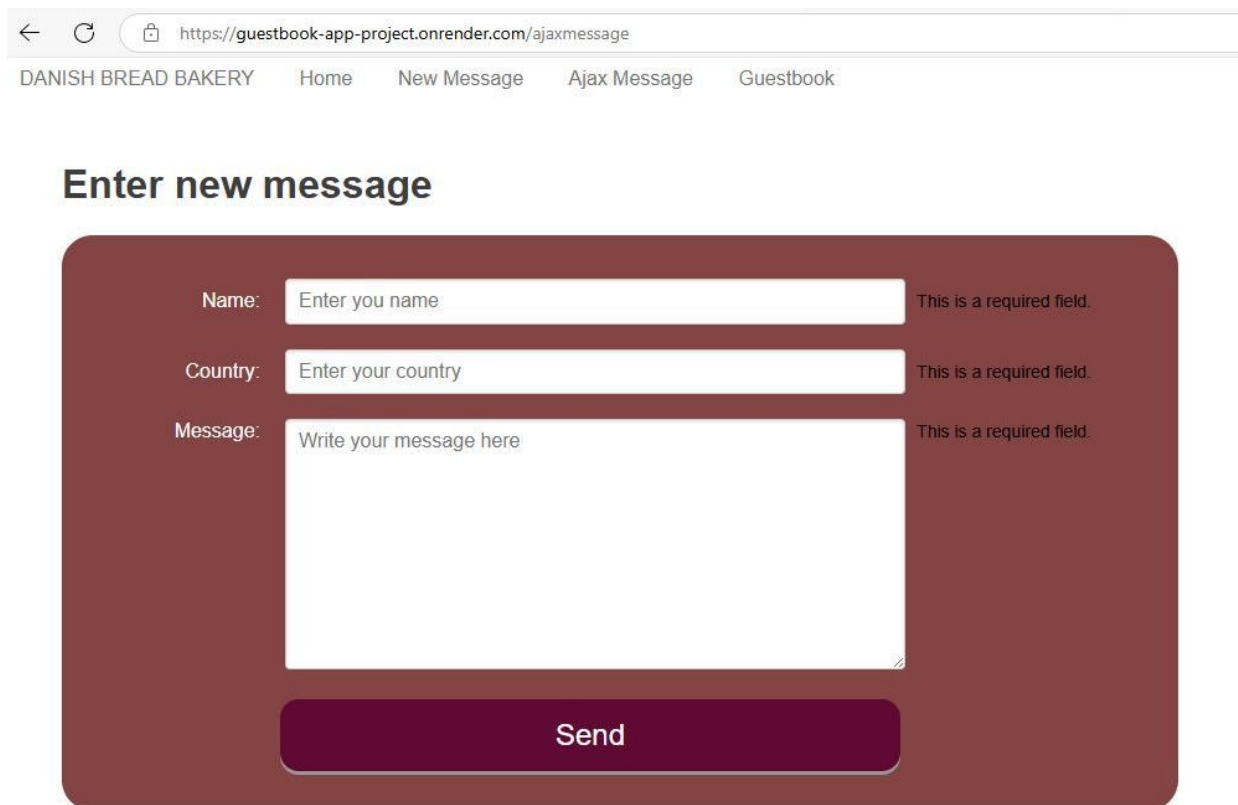
The application is hosted on Render, ensuring real-time access and consistent availability. The project's source code is hosted on GitHub, providing transparency and version control, while also allowing for easy collaboration and updates.

Technologies Used:

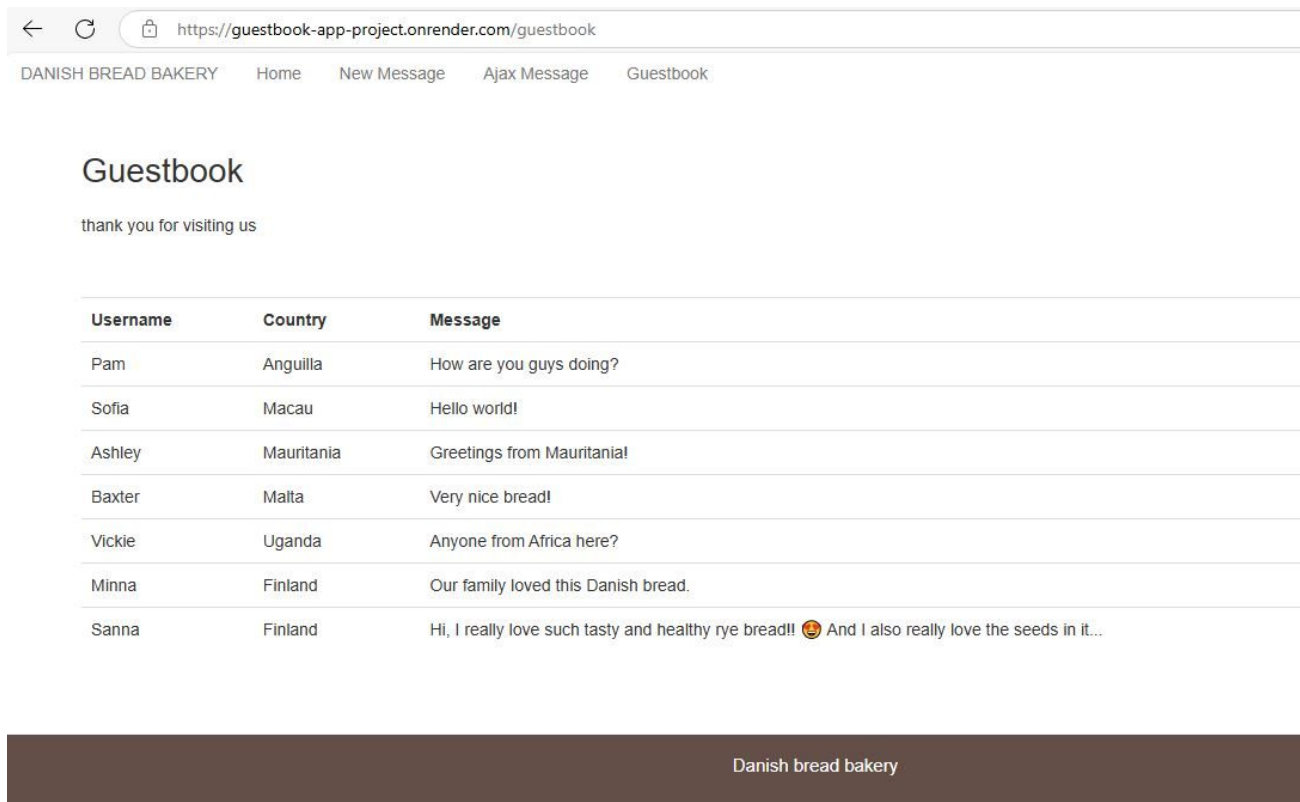
- Node.js & Express for backend development
- JavaScript (AJAX, native JS/jQuery) for dynamic functionality
- Bootstrap for front-end styling
- JSON for data storage

This project demonstrates my ability to build full-stack web applications, handle asynchronous operations, and deploy apps to cloud platforms.

Guestbook App Website View

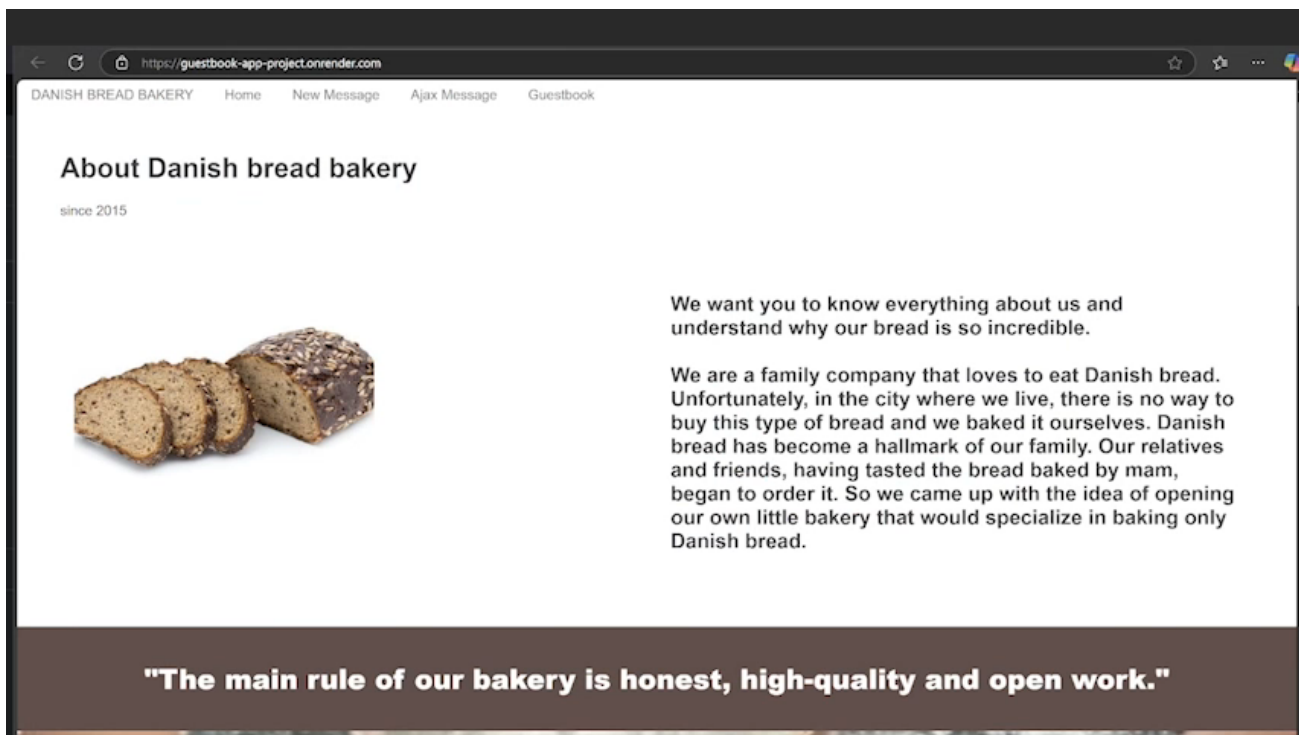


The screenshot displays a web browser window with the URL <https://guestbook-app-project.onrender.com/ajaxmessage>. The browser's navigation bar includes the site name "DANISH BREAD BAKERY" and a menu with links: "Home", "New Message", "Ajax Message", and "Guestbook". The main content area is titled "Enter new message" and features a dark red background. On this background, there is a form with three input fields: "Name:" with the placeholder "Enter your name", "Country:" with the placeholder "Enter your country", and "Message:" with the placeholder "Write your message here". Each input field is accompanied by a small red error message that reads "This is a required field.". Below the input fields is a large, rounded, dark red button labeled "Send".



View the deployed application on [Render](#) (it takes time to start Render!) and explore the source code on [GitHub](#):

Guestbook App on Render: Presentation



Challenges

One of the main challenges of this project was implementing the dynamic nature of the guestbook using AJAX. Ensuring that the data was submitted and displayed in real time without requiring a page reload required careful handling of asynchronous operations and seamless integration between the front-end and back-end.

Another challenge was managing data storage in the JSON file. As the guestbook entries were added, I had to ensure that the file was updated correctly and that the data persisted across server restarts without errors.

Additionally, styling the front-end to be responsive and user-friendly using Bootstrap required careful attention to detail. Ensuring that the design was clean and visually appealing on various screen sizes posed a challenge, especially when balancing functionality with aesthetic consistency.

Lastly, deploying the application on a cloud platform (Render) and ensuring it was accessible in real-time added complexity, as I had to ensure the app was running

smoothly and could handle multiple users interacting with it simultaneously.

Tools

- Visual Studio Code (VS Code)
- GitHub
- Render

[GitHub](#)[LinkedIn](#)

`viktoriya.mikhaylova@hotmail.com`

Copyright © 2026 Viktoriya Mikhaylova